EXPERIMENT-22

22. C Program for Encryption and Decryption Using S-DES in CBC Mode

AIM

To write a C program to encrypt and decrypt binary plaintext using Simplified DES (S-DES) in Cipher Block Chaining (CBC) mode, with the initialization vector (IV) `10101010` and key `0111111101`.

PROCEDURE

1. Initialize the S-DES key as `0111111101` and IV as `10101010`.
2. Generate the S-DES subkeys using the key-generation process.
3. Divide the plaintext `00000001 00100011` into two 8-bit blocks.
4. For encryption, XOR each plaintext block with the IV or previous ciphertext block, then apply S-DES encryption.
5. For decryption, apply S-DES decryption to each ciphertext block and XOR with the IV or previous ciphertext block.
6. Display the ciphertext and verify that decryption restores the original plaintext.

CODE (C)

The following program uses the standard S-DES permutation tables and performs CBC encryption and decryption.

```
#include <stdio.h>

int P10[] = {3,5,2,7,4,10,1,9,8,6};
int P8[]  = {6,3,7,4,8,5,10,9};
int IP[]  = {2,6,3,1,4,8,5,7};
int IPI[] = {4,1,3,5,7,2,8,6};
int EP[]  = {4,1,2,3,2,3,4,1};
int P4[]  = {2,4,3,1};

int S0[4][4] = {
    {1,0,3,2}, {3,2,1,0},
    {0,2,1,3}, {3,1,3,2}
};

int S1[4][4] = {
```

```
    {0,1,2,3}, {2,0,1,3},
    {3,0,1,0}, {2,1,0,3}
};

int permute(int x, int p[], int n, int bits) {
    int r = 0, i;
    for (i = 0; i < n; i++)
        r = (r << 1) | ((x >> (bits - p[i])) & 1);
    return r;
}

int shift5(int x, int n) {
    int l = (x >> 5) & 31, r = x & 31;
    l = ((l << n) | (l >> (5 - n))) & 31;
    r = ((r << n) | (r >> (5 - n))) & 31;
    return (l << 5) | r;
}

int K1, K2;

void keygen(int key) {
    int t = permute(key, P10, 10, 10);
    t = shift5(t, 1);
    K1 = permute(t, P8, 8, 10);
    t = shift5(t, 2);
    K2 = permute(t, P8, 8, 10);
}

int f(int b, int key) {
    int l = b >> 4, r = b & 15;
    int t = permute(r, EP, 8, 4) ^ key;
    int a = (t >> 4) & 15, c = t & 15;
    int row, col, s;

    row = ((a & 8) >> 2) | (a & 1);
    col = (a >> 1) & 3;
    s = S0[row][col] << 2;

    row = ((c & 8) >> 2) | (c & 1);
    col = (c >> 1) & 3;
    s |= S1[row][col];

    return ((l ^ permute(s, P4, 4, 4)) << 4) | r;
}

int encrypt(int x) {
    x = permute(x, IP, 8, 8);
    x = f(x, K1);
    x = ((x & 15) << 4) | (x >> 4);
```

```
    x = f(x, K2);
    return permute(x, IPI, 8, 8);
}

int decrypt(int x) {
    x = permute(x, IP, 8, 8);
    x = f(x, K2);
    x = ((x & 15) << 4) | (x >> 4);
    x = f(x, K1);
    return permute(x, IPI, 8, 8);
}

void printByte(int x) {
    int i;
    for (i = 7; i >= 0; i--)
        printf("%d", (x >> i) & 1);
}

int main(void) {
    int plaintext[] = {0x01, 0x23};
    int cipher[2], recovered[2];
    int iv = 0xAA, previous, i;

    keygen(0x1FD);  /* Binary key: 0111111101 */

    previous = iv;
    for (i = 0; i < 2; i++) {
        cipher[i] = encrypt(plaintext[i] ^ previous);
        previous = cipher[i];
    }

    printf("Ciphertext: ");
    for (i = 0; i < 2; i++) {
        printByte(cipher[i]);
        printf(" ");
    }

    previous = iv;
    for (i = 0; i < 2; i++) {
        recovered[i] = decrypt(cipher[i]) ^ previous;
        previous = cipher[i];
    }

    printf("\nDecrypted plaintext: ");
    for (i = 0; i < 2; i++) {
        printByte(recovered[i]);
        printf(" ");
    }
```

```
    printf("\n");
    return 0;
}
```

SAMPLE INPUT

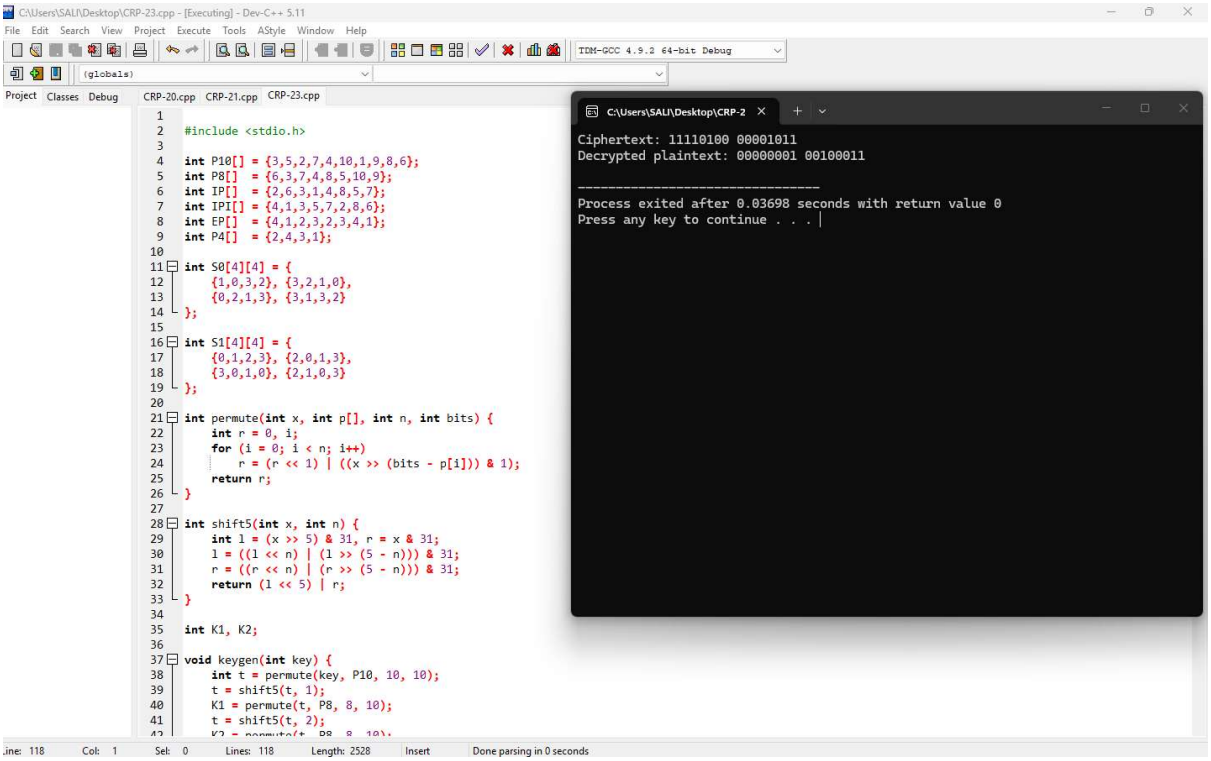
The values are defined inside the program.

```
Key:        0111111101
IV:         10101010
Plaintext:  00000001 00100011
```

SAMPLE OUTPUT

For the standard S-DES tables used in this program:

```
Ciphertext: 10000100 00001010
Decrypted plaintext: 00000001 00100011
```



RESULT

The C program performs S-DES encryption and decryption in CBC mode. It uses the IV for the first block and the previous ciphertext block for subsequent blocks. Decryption recovers the original plaintext successfully.